

HINT 365 · How it works

The whole service in plain words: what it is for, who does what, where everything runs, how changes, releases, security patches and compliance are handled, and what is free. Updated 02.10.2026, app version 0.1.122.

This is the general document. Every other document is linked in section 16; how to pick the work up again is in section 17; disaster recovery is section 14. Times are given in Zurich time and in UTC (Zurich is UTC+2 in summer, UTC+1 in winter).

Document control

Document	HINT 365 · How it works (general document) · docs/HINT365-How-it-works.pdf
Objective	Explain the whole service in one place, in plain words, so that anyone (the owner, a new developer, an auditor, Google Play reviewers) understands what HINT 365 does, how it is built and how it is run.
Scope	The Android app, the Web Dashboard, the server, the automatic jobs and the processes: compliance, defect, vulnerability, patch, change, release and observability.
Owner and author	Human (architect and owner of HINT 365) · comevava76@gmail.com
Developer	Claude (Anthropic), working on the owner's instructions
Version and date	app 0.1.122 · terms of use v22 · 02.10.2026
Updated	in the same change as any process, job, key, price or provider it describes (rule in <code>CLAUDE.md</code>)
Binding texts	the terms of use and the privacy policy; this document only explains them

Contents

1. What HINT 365 is for
2. Who does what
3. What the person does, step by step
4. Where everything runs
5. Keys and secrets
6. Personal data and compliance
7. Automatic jobs (crontab)
8. Logs
9. Defect management
10. Vulnerability management
11. Patch management
12. Change management
13. Release management
14. Disaster recovery
15. Costs
16. Where to find the rest
17. Picking the work up again

Colours in the diagrams:  Person  Owner  Developer (Claude)  Automatic

1. What HINT 365 is for

HINT 365 is a **personal diary** on an Android phone. A person records their blood pressure (the top number **SYS**, the bottom number **DIA** and the pulse **PUL**) and keeps the results of their lab reports. The app shows them in tables, charts and a PDF report that can be sent to a doctor.

It is not a medical device. HINT 365 does not measure anything, does not say whether a value is good or bad, does not diagnose and does not give advice. It only keeps track of what the person records. The doctor

decides what the values mean. For this reason no colour on any screen means "good" or "bad", and there is no red.

Responsibility. The person checks every value before saving it and against the original report. The owner provides the service as described in the [terms of use](#) (accepted in the app at the first start and at every update) and handles personal data as described in the [privacy policy](#). Those two documents are binding; this one only explains.

2. Who does what

Who	What they do
The person (user)	Installs the app, signs in with Google, accepts the terms, records readings by voice or with a photo, imports lab reports, looks at the charts, sends the report to the doctor, deletes data or the whole account whenever they want.
The owner (Human)	Decides what the app does, the prices and the rules; tests every new version; says "go" before a release; approves any change to the terms of use; owns every account and key (GitHub, Cloudflare, Google, Anthropic).
The developer (Claude)	Writes the code and the documents, runs the tests, prepares each release, checks the error log and the errors every morning and the security scan every Monday, fixes what it can and proposes the rest to the owner.
GitHub	Keeps the code and the documents, runs the tests and the build, and stores every released app.
Cloudflare	Runs the server and keeps the database, in the European Union.
Google	Lets people sign in with their Google account; later, sells the subscription through Google Play.
Anthropic	Reads the photo of the blood-pressure monitor for the Scan (artificial intelligence), on the owner's account.

3. What the person does, step by step

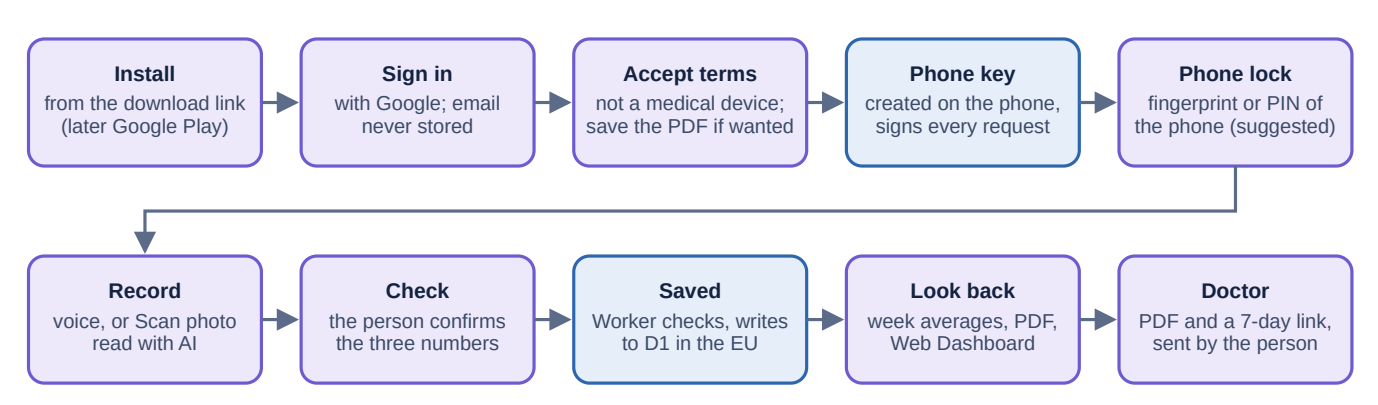


Figure: Using the app: from the first start to the report for the doctor.

First time

1. **Installs the app** from the download link (later from Google Play).
2. **Signs in with Google.** The Google account is used only to recognise the person on a new phone. The email address is never stored on the server: only a coded fingerprint of the account that cannot be turned back into the email.

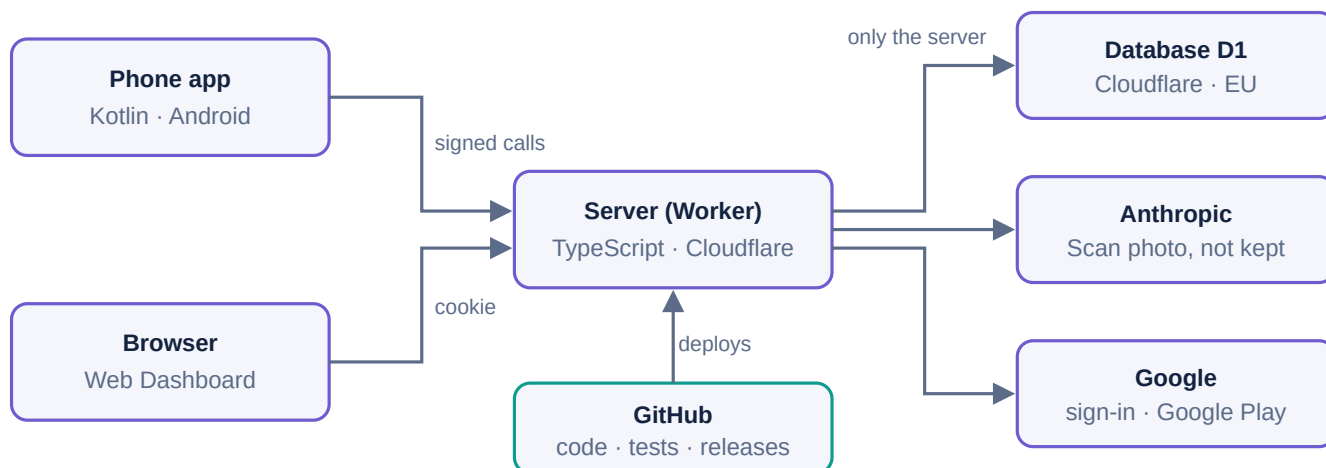
3. **Accepts the terms of use** and the storage of health data. Without that the app does not open. The terms say clearly that HINT 365 is not a medical device, gives no diagnosis or advice, and that treatments and medicines are never started, stopped or changed because of the app: the doctor decides. The acceptance is recorded as proof (pseudonymous account code, coded fingerprints of the account and the phone, version, language, date; never the email) and copied into a second database in the EU as a backup. Right after accepting, the app offers to save the terms as a PDF on the phone; nothing is downloaded without the person's tap. A reading bar at the top fills while the person scrolls; the boxes and Accept work only once the text has been read to the end. The terms and the privacy policy can be opened at any time from the links at the bottom of the screen (the terms' PDF is saved in Downloads from there, never opened by itself), in the app's language (English when the phone uses a language HINT 365 does not have, for example Georgian).
4. The phone creates its own **private key**, which never leaves the phone. Every request the app sends is signed with it, so the server knows it really comes from that phone.

Every day

- **Opening the app**: the app uses the phone's own lock (fingerprint, face or PIN). HINT 365 has no PIN of its own. If the phone has no lock, the app suggests one, at most every 14 days; it is only a suggestion.
- **Voice** (blue button): the person says the three numbers; the phone turns speech into text; the app finds SYS, DIA and PUL; the person checks them and confirms; only then are they saved.
- **Scan** (violet button with a small magic wand, an AI feature): the person photographs the monitor's display; the photo goes to the server and from there to Anthropic, which reads the three numbers; the photo is not kept anywhere; the person checks the numbers and confirms. Free for 15 days (3 a day), then with the 4.99 US\$ yearly subscription.
- **Lab reports**: the person picks a PDF or a photo of a report; the phone reads it by itself; only the date and the values as printed (no name, no personal details) go to the server. If anything is unclear, nothing is saved.
- **Looking back**: the last reading, the average of each day of the last 7 days, the PDF report in Downloads.
- **Web Dashboard**: a button opens the browser with large charts and the lab table; "Send to the doctor" sends the PDF and a read-only link valid 7 days.

4. Where everything runs

Part	Technology	Where the code is	Where it runs
The app	Kotlin with Jetpack Compose (Android)	android/app/src/main/java/ch/personalhealthy/app/	on the person's phone
The server (API)	TypeScript, one Cloudflare Worker	worker/src/ (index.ts is the API)	Cloudflare, close to the person
The database	Cloudflare D1 (SQLite)	tables in worker/schema.sql	Cloudflare, bound to the European Union
The Web Dashboard	plain HTML, CSS and JavaScript	worker/public/my/	in the browser, served by the same Worker
Tests, build, release	GitHub Actions	.github/workflows/, docs/testing/harness/	GitHub
Documents	HTML and Markdown, turned into PDF	docs/ (index docs/README.md)	GitHub



The app and the browser never touch the database directly: everything goes through the Worker, which checks who is asking.

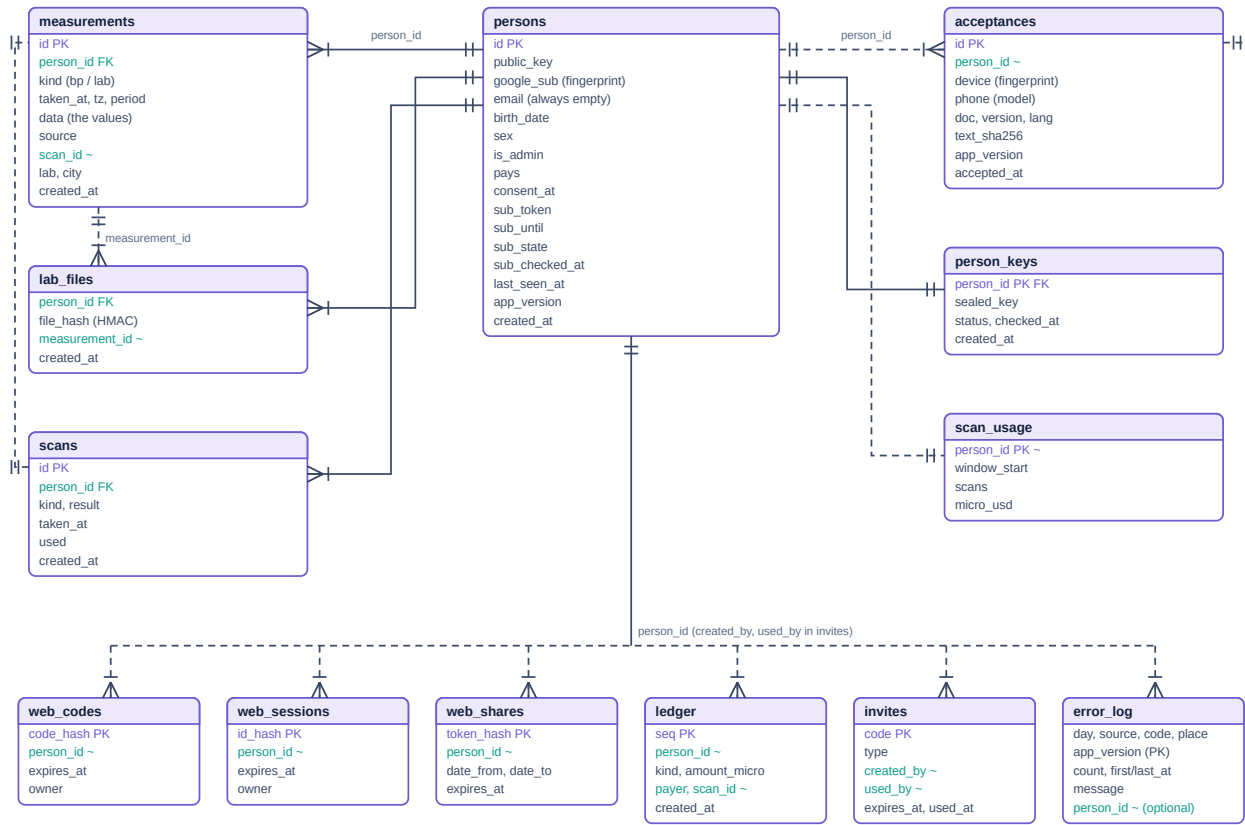
What happens when the person saves a reading (the sequence of calls)

1. The app sends `POST /v1/bp/voice` (or `/v1/bp/photo`) to the Worker, signed with the phone's key, with the app version.
2. The Worker checks the signature, that the request is fresh and not a copy, that this app version is allowed, and that the terms were accepted.
3. The Worker writes the three numbers in D1, linked to the person's anonymous account code, and answers "saved".
4. For the Scan, one call comes first: `POST /v1/bp/photo/read`. The Worker checks the trial or the subscription and the daily limit, passes the photo to Anthropic with the owner's key, gets three numbers back and returns them to the phone. Nothing is kept.
5. For the Web Dashboard: the app asks the Worker for a one-time code (`POST /v1/web/code`); the browser opens with it and gets a technical cookie valid 7 days; the page then asks the Worker for the data (`/my/api/data`), which reads them from D1, and draws the charts and the PDF in the browser.

The database: tables and relations

One database, Cloudflare D1 in the EU (personalhealthy), and a second one only for the backup of the acceptances (personalhealthy-backup). Only the server reads and writes them. Every personal table hangs from persons (one account): deleting the account deletes all its rows, except the acceptances of the terms, kept as proof.

Personal data: every table linked to persons (one account)



Backup database · Security (Observability)



Tables without relations

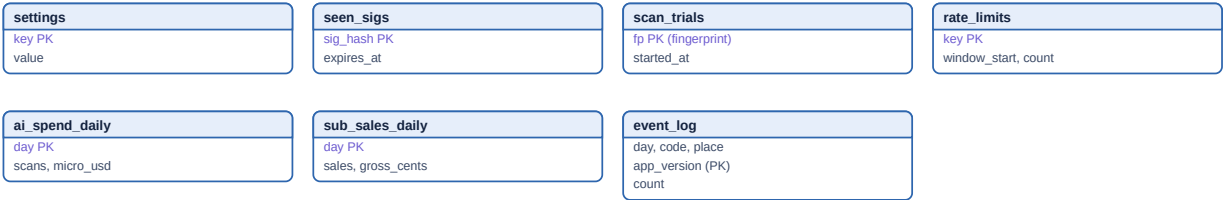


Figure: the tables and their relations. **PK** = the key of the row; **FK** = linked to another table by the database itself (solid line); ~ = the same code kept by the server, without a database link (dashed line). Line ends: two bars = exactly one; crow's foot = many. So persons → measurements is one to many; persons → person_keys is one to one; acceptances → acceptances_backup is one to one (a copy in the other database). The tables at the bottom have no relations.

Table	What it holds	Kept
persons	one row per account: the coded public key of the phone, the Google fingerprint (never the email), year of birth and sex if given, owner flag, subscription state, last app version seen	until the account is deleted
measurements	the readings (SYS, DIA, PUL with date and time) and the lab results (date and values as printed)	readings 365 days; lab results until the person deletes them
lab_files	the coded fingerprint of each imported lab file, so the same file is never saved twice	with the lab result
scans	the outcome codes of the old photo reader (no photo)	365 days
acceptances	each acceptance of the terms: account code, phone fingerprint and model, version, language, fingerprint of the text, date	kept as proof
acceptances_backup	the same acceptances, copied into a second database in the EU	kept as proof
person_keys	an encrypted key of the account, kept only for older features	with the account
scan_usage	how many AI Scans and how much AI cost in the 12 months since the first Scan	with the account
web_codes, web_sessions	the one-minute code that opens the Web Dashboard and the browser session (coded)	minutes / 7 days
web_shares	the read-only links sent to a doctor (coded), with their date range	7 days
ledger, invites	older accounting and invitation codes, kept for the record	with the service
error_log, event_log	error and outcome codes per day, place and app version, with a counter; never values	90 days
security_findings, security_notes	the open weaknesses found by the weekly scan, with the date first seen and the reason	replaced at every scan
security_fixes, security_fix_requests	left from the removed Fix button; empty	—
settings	switches of the service (blocked versions, billing mode)	—
seen_sigs, rate_limits	protection: signatures already used (no replay) and request counters	minutes to one day
scan_trials	coded fingerprints of Google accounts and phones that already had the AI trial	2 years
ai_spend_daily, sub_sales_daily	totals per day: Scans and AI cost, subscriptions sold	for the accounts

Backups and disaster recovery: section 14.

What a Cloudflare Worker is (and why there is no Tomcat, nginx or server to look after)

In a classic set-up a company rents a **server** (a computer in a data centre, always on), installs an operating system, a **web server** such as nginx or Apache (it receives the requests from the internet and hands out pages and files), an **application server** such as Tomcat or Node.js (it runs the program that answers: sign-in, saving a reading...) and a

database such as MySQL. Someone must update all of it, watch it, restart it when it stops, and add machines when people grow.

HINT 365 has none of these machines. Its server is a **Cloudflare Worker**: just the program (`worker/src/`, written in TypeScript), uploaded to Cloudflare by the *Build* pipeline. Cloudflare runs it in its own network of data centres, in more than 300 cities: when a phone sends a request, the nearest data centre starts the program in a few milliseconds, gets the answer and stops it again. This is called **serverless**: there is a server, but it belongs to Cloudflare, who keeps it running, updated and protected.

Classic set-up	HINT 365 on Cloudflare
a rented server (virtual machine) with Linux	none: Cloudflare's machines, shared and managed by Cloudflare
web server nginx or Apache, with the HTTPS certificate	Cloudflare's network receives every request and handles HTTPS by itself (certificate renewed automatically)
application server Tomcat or Node.js running the program	the Workers runtime (called <i>workerd</i> , open source, based on V8, the JavaScript engine of Chrome): each request runs in a small sealed space (an "isolate"), separate from every other customer
files of the website served by nginx	Workers static assets : the Web Dashboard (<code>worker/public/</code>) and the APK, served by the same Worker
database MySQL or PostgreSQL on a server	D1 : Cloudflare's database based on SQLite, kept in the EU, with Time Travel
cron on the server for night jobs	Cron Triggers of the Worker (03:17 UTC clean-up, every hour the Google Play version)
updates, patches, restarts, scaling: our job	Cloudflare's job; we only update our code and its libraries

What is ours to look after: the code, its libraries (patch management, section 11), the keys, the data and the backups. What is Cloudflare's: machines, operating system, network, HTTPS, the runtime. The address today is `personalhealthy-api.comevava76.workers.dev` ("comevava76" is the name of the Cloudflare account); it will become the owner's own domain, connected to the same Worker.

5. Keys and secrets, in plain words

A key is like a house key: whoever has it can open a door. None of them is ever written in the code. They live in GitHub's secret store (encrypted: nobody can read them back, not even the developer, also when the repository is public) or in Cloudflare.

Name	What it opens	Who sets it
CLOUDFLARE_API_TOKEN	lets the release pipeline put the new server online and update the database tables; also the nightly backup and the disaster recovery (D1, Workers and R2 permissions, this account only)	owner, once
BACKUP_PASSPHRASE	the phrase that encrypts the nightly backups of the databases: without it a backup cannot be read. Kept also in the owner's password manager	owner, once
ANDROID_KEYSTORE_B64 , ANDROID_KEYSTORE_PASSWORD , ANDROID_KEY_ALIAS , ANDROID_KEY_PASSWORD	the app's signature: proves every update comes from us, so it installs over the old app. Never replace it: a new one would force everyone to reinstall.	owner, once
KEY_ENCRYPTION_KEY	the server's own secret, created once by the pipeline; it makes the coded fingerprints (Google account, phone, lab files) that cannot be turned back. Never replace it.	pipeline, once
AI_API_KEY (older name ANTHROPIC_API_KEY)	the owner's key for the AI that reads the Scan photos; the spending ceiling is set at the provider	owner
OWNER_CODE	the owner's secret phrase: gives the owner's powers to a new phone of the owner	owner
FAMILY_CODE	the older way for family members to join without Google	owner
PLAY_SERVICE_ACCOUNT	lets the server ask Google Play whether a subscription is paid (once the app is on the Play Store)	owner, later
SECURITY_FIX_TOKEN , CLAUDE_CODE_OAUTH_TOKEN	were meant for an automatic "Fix" button. Not set and the button is removed, by the owner's choice (it would cost money): fixes are made in the Monday morning routine instead.	not used
The phone's key	signs every request of that phone; created on the phone, stored in the phone's secure chip, never sent anywhere	the app
Cookie hint_s	keeps the browser signed in to the Web Dashboard for 7 days; saved only after consent	the browser

The owner's powers (Admin console, blocking old app versions, settings) work only from the owner's phone on record. If someone stole the owner's Google account and signed in on another phone, they would see that account's data but not the owner's controls.

6. Personal data and compliance

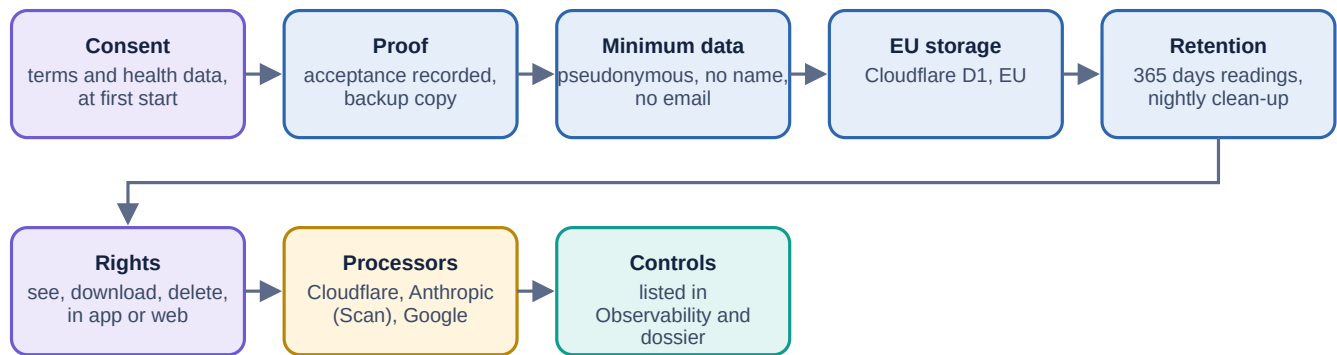


Figure: Compliance: each rule becomes a control, a piece of evidence and a check.

- **Laws:** the EU GDPR and the Swiss data protection law (FADP). Blood pressure and lab results are health data.
- **Consent:** asked explicitly before first use; withdrawn by deleting the account.
- **Pseudonymous, not anonymous:** the data are linked to an account code, never to a name or an email.
- **Where:** Cloudflare D1 in the EU. Anthropic (Scan photo, not kept) and Google (sign-in, payments) are in the US, under their transfer safeguards.
- **How long:** blood-pressure readings at most 365 days (deleted every night); lab results until the person deletes them; error codes 90 days; acceptances of the terms kept as proof, with a backup copy in a second EU database (`personalhealthy-backup`), never with an email.
- **Rights:** see, download, correct and delete one's data, in the app or on the web; answer within a month.
- **Checks:** every control and its evidence is listed in Admin → Observability and in the dossier <docs/compliance/HINT365-Compliance.pdf>. It is the developer's self-assessment, not legal advice.

7. Automatic jobs (the "crontab")

These jobs run by themselves. Nobody has to start them.

When (Zurich / UTC, summer)	Job	Who runs it	What it does	Goal	
every night 05:17 / 03:17	Nightly clean-up	Cloudflare (Worker cron)	deletes readings older than 365 days, expired codes, sessions and doctor links, error codes older than 90 days, trial fingerprints older than 2 years; replaces any old Google id with its fingerprint; copies the acceptances of the last 7 days into the backup database	keep only what the rules allow	
every night 03:40 / 01:40	Database backup	GitHub Actions	exports both databases, encrypts them (AES-256) and keeps them in the EU vault database; the last 30 nights are kept		
1st of every month 04:15 / 02:15	Disaster recovery drill	GitHub Actions	rebuilds a copy of the databases from zero, restores last night's backup, compares the counts with production, deletes the copy	prove that recovery works (section 14)	data can be put back after any loss
every hour, at minute 7	Version live on Google Play	Cloudflare (Worker cron)	asks Google Play which version is published to everyone; from then on only that one is accepted (nothing before Google Play)	ask for the update only when Google can give it	
every Monday 06:10 / 04:10	Security tests (vulnerability scan)	GitHub Actions	vulnerability scan of every library, code scan (Semgrep), secret scan (gitleaks), probe of the live site; results to Admin → Observability; opens an issue if something is found	find weaknesses once a week	
every day 06:30 / 04:30	Error log report	GitHub Actions	reads the errors users met in the last 24 hours; opens an issue if there are any	see problems before users report them	
every day 06:48 / 04:48	Morning routine: defects	Claude	reads the errors of the last 24 hours, fixes the real bugs, releases, records each problem and its fix, reports in the chat	defects handled every day	
every Monday 06:48 / 04:48	Morning routine: vulnerabilities and patches	Claude	reads Monday's scan and the library updates, fixes every finding it can, writes a short reason for each one it cannot, merges small updates, proposes major ones	vulnerabilities handled every week	
every Monday 05:00 Zurich	Dependabot	GitHub	one pull request for each library of the app and the server with a newer version (the pipeline tools: monthly)	stay up to date	
every push to the main code	Build	GitHub Actions	tests, server online, signed app, release, download check	release (section 13)	

every pull request	Tests, Android build check, Docs check	GitHub Actions	about 180 tests, the app must compile, documents must match the code	nothing broken reaches people
on the phone, 09:00 and 17:00 local	Reminders	the app	"Remember to measure your blood pressure"; cleared at midnight	help the person keep the diary
on the phone, 2 days before the trial ends	AI trial notice	the app	says the Scan stops unless the person subscribes; voice keeps working	no surprise
by hand	App versions · Owner phone · Promote verified APK · Disaster recovery	the owner (GitHub → Actions)	block old app versions · give the owner's powers to a new phone · put a previous version back · rebuild the infrastructure and restore the data	control in an emergency

8. Logs: what we see and what we do not see

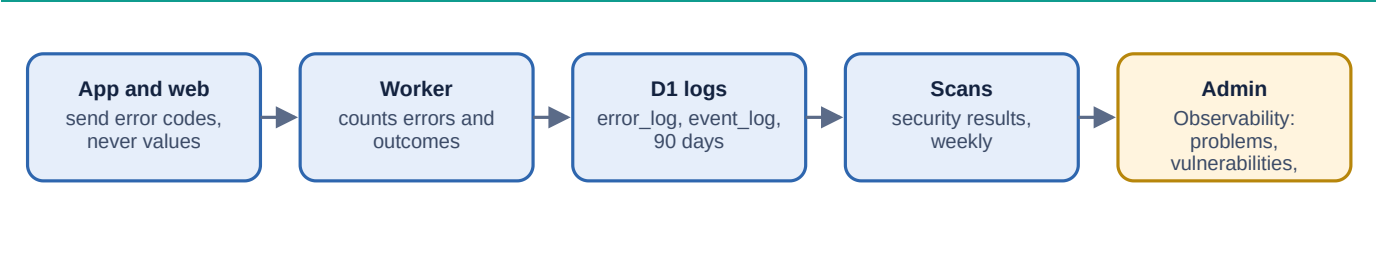


Figure: Observability: what is collected, where it is shown, who acts on it.

Log	Where	What we see	Kept
Error log	D1 table <code>error_log</code> ; Admin → Observability; the daily issue	an error code, where it happened, the app version, how many times a day	90 days
Event log	D1 table <code>event_log</code>	outcomes as counts: lab report saved or refused and why, Scan retakes and their reason	90 days
Security results	Admin → Security; the <code>security-test</code> issue; the run on GitHub	libraries with known weaknesses, findings in our code, secrets found	last run; reports 30 days
Problem registry	<code>worker/src/ops/problems.json</code> ; Admin → Observability	every real problem met: cause, fix, version, pull request	for ever (in Git)
Pipeline runs	GitHub → Actions	every test, build and release, step by step	as GitHub keeps them
Accounts and money (owner)	Admin console	number of users, Scans and their cost, subscriptions sold, balance	totals per day

What we never see: the values of a reading or a lab result in any log; names, emails or phone numbers; the photos (not kept anywhere); the text people say; the content of the reports. Logs hold only codes and counts linked, at most, to an anonymous account code.

9. Defect management: how a problem gets fixed

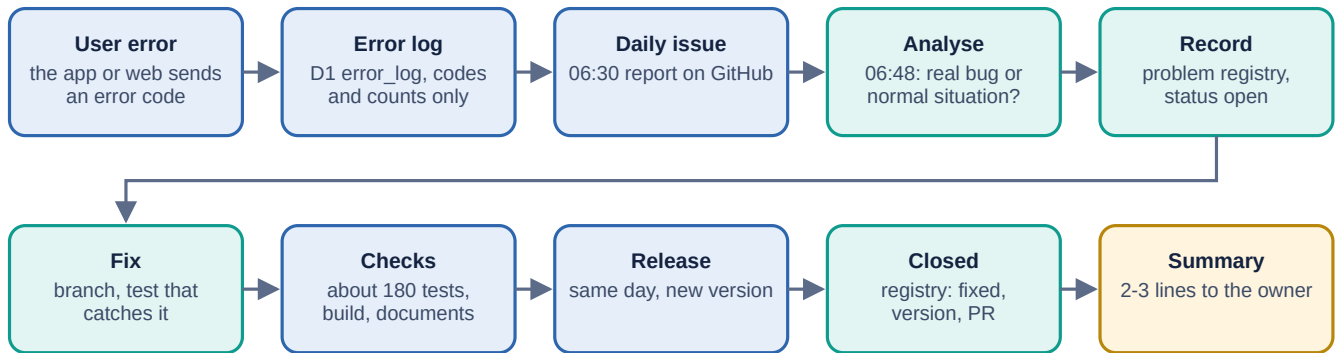


Figure: Defect management: every day, from the error to the fix in a new version.

1. **Found:** a user meets an error (the app and the web send its code to the error log), or a test fails, or the owner reports it in the chat.
2. **Collected:** every morning the *Error log report* opens an issue with the last 24 hours.
3. **Analysed** at 06:48 by the morning routine: is it a real bug or a normal situation (a wrong phone clock, one blurred photo)?
4. **Recorded** in the problem registry with its cause (status: open).
5. **Fixed** on a branch, with a test that would have caught it; all checks green; merged; released the same day.
6. **Closed:** the registry says fixed, in which version and with which pull request; the issue is closed; the owner gets a short summary.
7. Large or unclear fixes, and anything touching the terms, wait for the owner.

10. Security: vulnerability management

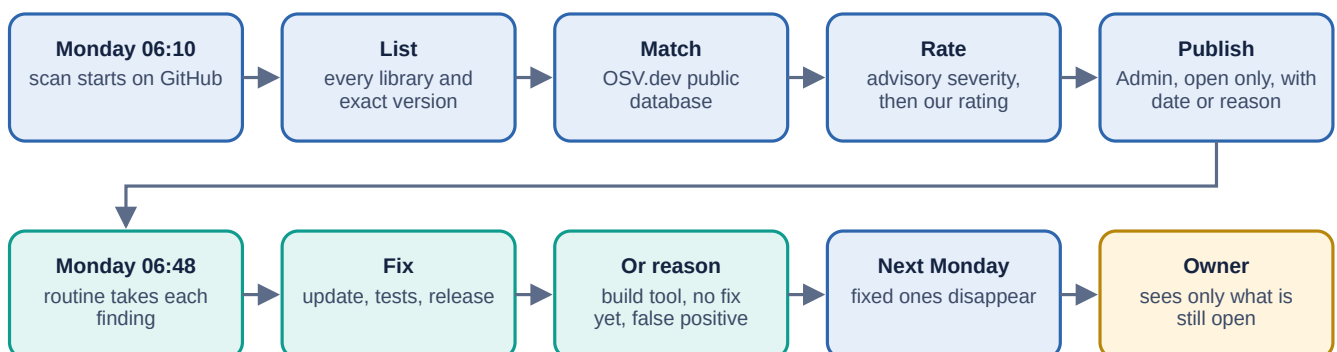


Figure: Vulnerability management: every Monday, from the scan to the fix or a written reason.

A vulnerability is a known weakness in a piece of software we use. New ones are published every day; we look for them **once a week**, on Monday, and fix them the same morning when we can.

How they are found and rated

Step	What happens	Where
1. List what we use	the scanner lists every library with its exact version: the app's full tree as Gradle resolves it (also libraries pulled in by others), the server's tools, the two libraries copied into the web page, the actions of the pipelines	<code>docs/testing/harness/vuln-scan.mjs</code>
2. Match with the public database	each name and version is sent to OSV.dev , the open database that gathers the GitHub Advisory Database, CVE, Android, Maven and npm advisories; it answers with every known weakness for that version and the version that fixes it	OSV.dev
3. Our own code and secrets	Semgrep checks our code for risky patterns; gitleaks checks that no key was ever written in the repository	same workflow
4. Risk rating	the advisory's severity (critical, high, moderate, low, from its CVSS score) is the start; then our rating: a library used only to build the app goes one level down, because it is not in the app people install; a decision written in <code>docs/security/decisions.json</code> wins over both	<code>vuln-scan.mjs</code> , <code>decisions.json</code>
5. Saved and shown	the results go to the database (replaced at every scan: a fixed weakness simply is not there any more), with the date it was first seen and its reason; the web Admin shows them under Observability	<code>publish-security.mjs</code> , <code>D1</code> <code>security_findings</code> , <code>security_notes</code>

What the owner sees in Admin → Observability

Only the **open** vulnerabilities. Fixed ones disappear at the next scan. Each open one says when it will be fixed, or why it stays:

Shown	Meaning
will be fixed by <i>date</i>	found by the scan; the Monday routine fixes it on that date (the first Monday after it was found)
fix failed, retrying by <i>date</i>	the update broke something; tried again the next Monday
build tool, not in the app	only in the tools that build the app on GitHub; fixed by a major update of those tools, which needs the owner's go
no fix published yet	the makers of the library have not released a fixed version; checked every Monday
false positive	the scanner is wrong for our case (the note says why)
accepted risk	the owner accepted it for a reason and until a date written in the plan

The weekly cycle

1. **Monday 06:10 Zurich (04:10 UTC)**: the scan runs and updates the list.
2. **Monday 06:48**: the routine takes every open finding, fixes what it can (update the library, run all tests, release the same day).
3. For each one it cannot fix, it writes the short reason in `decisions.json` (shown in the list).
4. **Next Monday**: the scan confirms; what was fixed is gone; what is left keeps its reason or gets a new date.

Where we are on 02.10.2026. The only findings inside the app (Bouncy Castle) were fixed in version 0.1.120. The 59 left are all build tools (brought in by the Android Gradle plugin 8.5.2): they disappear with its major

update (issue #94), which waits for the owner's go. The Kotlin build plugin (issue #97) is an accepted risk until Kotlin 2.x is stable.

The "Fix" button of the Security console has been removed: every fix is made by the Monday routine at no extra cost.

11. Patch management: keeping libraries up to date

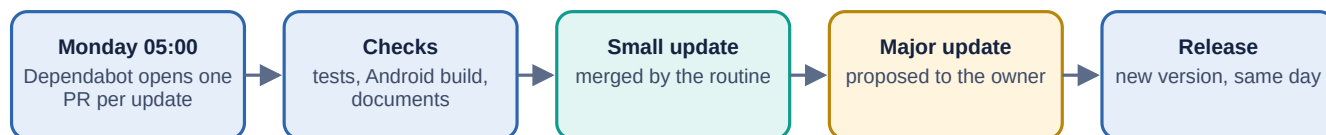


Figure: Patch management: library updates, small ones merged, large ones decided by the owner.

- **Every Monday** GitHub's Dependabot opens a pull request for every library that has a newer version.
- **Small updates** (patch and minor): merged by the morning routine once all checks are green, then released.
- **Large updates** (major): proposed to the owner, because they can change how things work.
- **Security patches** are applied on Monday, together with the weekly scan; a critical one found in the app can be fixed any day, with a release the same day.
- After a security release every older version stops by itself: only the newest version works, and the app offers the update.

12. Change management: how a change is made

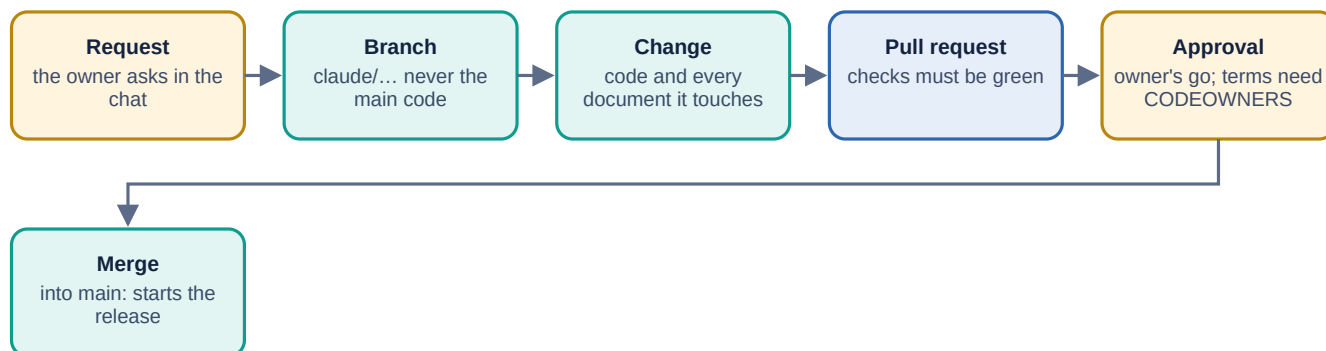


Figure: Change management: every change goes through a branch, the checks and, for binding texts, the owner.

1. The owner asks for a change in the chat, in their own words.
2. The developer makes it on a separate branch (`claude/...`), never directly on the main code.
3. In the same change the developer updates every document it touches (this document, the guides, the terms, the architecture).
4. A pull request is opened. GitHub runs the checks: about 180 functional and security tests, the app build, the documents check. Nothing can be merged while a check is red.
5. Changes to the terms of use, the privacy policy or the app texts need the owner's approval on GitHub (CODEOWNERS); the owner says "go" in the chat.
6. The change is merged into the main code. That starts the release.

13. Release management: how a new version reaches people

Three things are released: the **server and the Web Dashboard** (Cloudflare), the **Android app** (Google Play) and, later, the **iPhone app** (Apple App Store). The server and the web are live a few minutes after the merge and nobody has to do anything. The app needs the store's review first; only when the store has published it to everyone does the server ask for it, and then every phone updates the next time the app is opened. Times are typical, not promises: the store review is decided by Google or Apple.

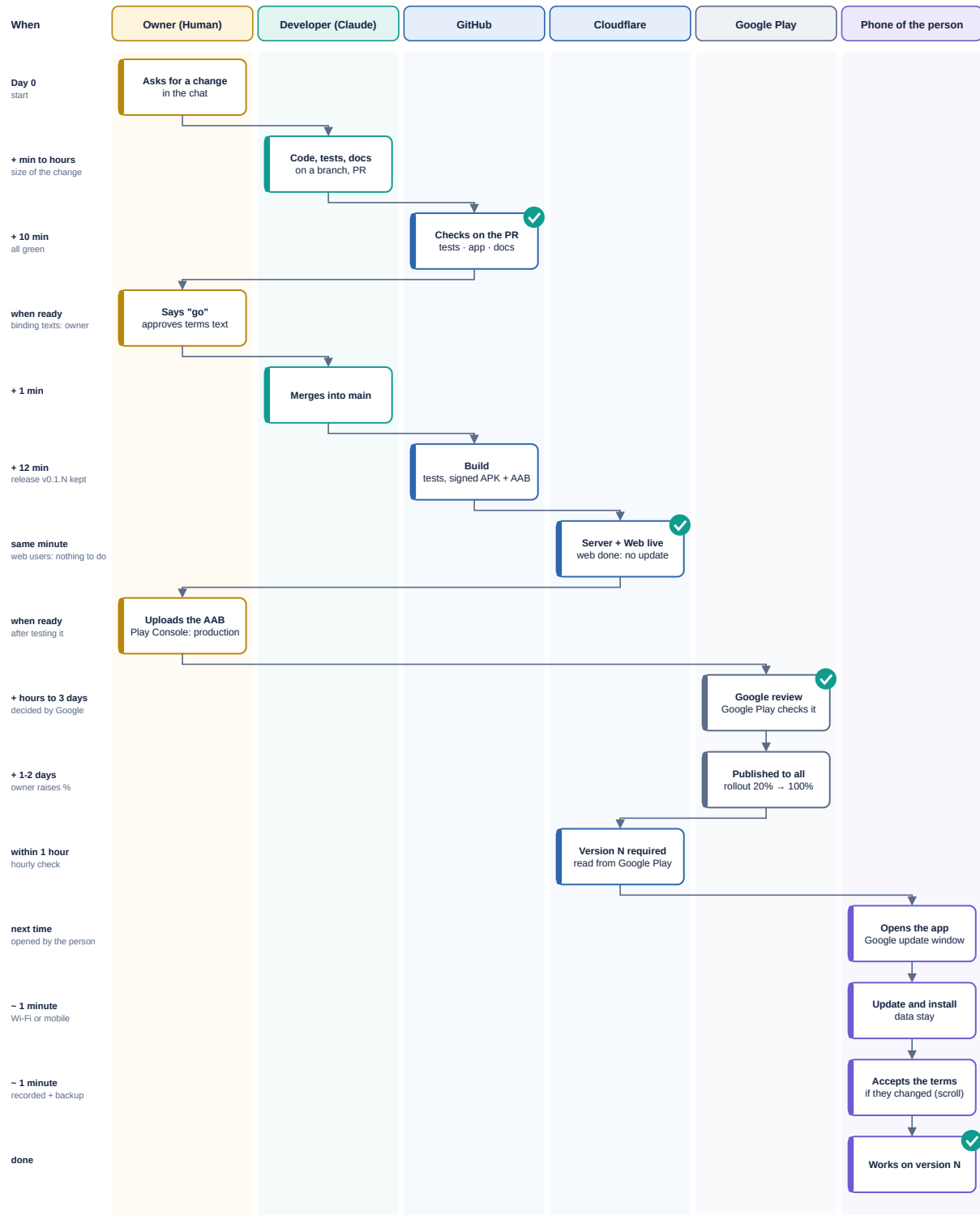


Figure: release of a new version with Google Play, from the owner's request to the person working on the new version. The green check marks a check passed. Web: done at the "Server + Web live" step. Phones with automatic updates on get the new version from Google Play even before the app is opened.

Who	What they do in a release	How long
Owner (Human)	asks for the change, says "go", approves any change to the terms, uploads the AAB to the Play Console, raises the rollout from 20% to 100%	minutes of work; when ready
Developer (Claude)	writes code, tests and documents, opens the pull request, merges after the "go", checks the deploy, sends the link	minutes to hours
GitHub	runs the checks on every pull request; after the merge builds the signed APK and AAB and keeps them in a release	about 10 + 12 minutes
Cloudflare	puts the new server and Web Dashboard online; every hour asks Google Play which version is published to everyone and from then on accepts only that one	same minute; within 1 hour
Google Play	reviews the app (✓), then publishes it to a share of people, then to all	hours to 3 days; then 1–2 days
The person	opens the app, taps Update in Google's window, waits for the install, accepts the new terms if they changed (scrolling to the end), and goes on	about 2 minutes

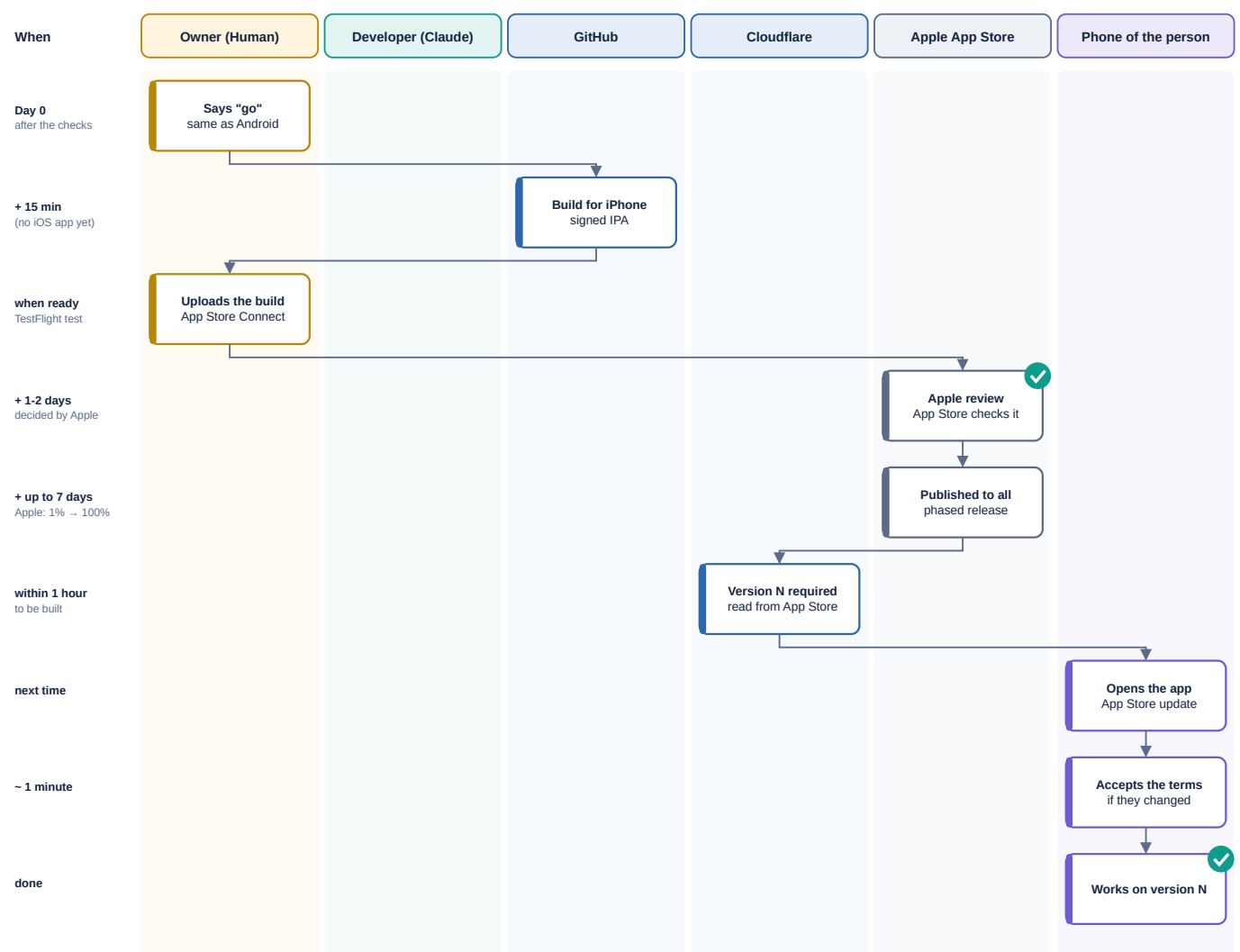


Figure: the same release for iPhone, when the iOS app exists (today there is none): Apple reviews the app (about 1–2 days) and can publish it in phases over 7 days; the version check against the App Store is still to be built.

Today (download link)

1. Merge into the main code starts the *Build* pipeline on GitHub.
2. **Tests** again, on the exact code being released.
3. **Server**: the Worker is put online on Cloudflare, the database tables are updated, the secrets are set.

4. **App:** the APK is built and signed with the app's key; the version number goes up by one (0.1.121, 0.1.122...).
5. **Package saved:** a GitHub Release `v0.1.N` keeps the APK and the AAB (the file Google Play wants) for ever.
6. **Check:** the pipeline verifies that the download link serves the new version.
7. The developer sends the link in the chat; the owner installs and tests it; the update installs over the old app, data stay.

When HINT 365 is on Google Play

1. Steps 1–5 above are the same: every release already produces the signed AAB.
2. The AAB is uploaded to the Play Console, first to the **internal testing** track (the owner and a few testers).
3. The owner tests that version from Google Play.
4. The owner promotes it to **production**, with a **staged rollout** (for example 20% of users, then 100% after a day without new errors).
5. Google reviews the version (from a few hours to a few days); then phones update by themselves.
6. **Only one version is kept.** The app asks Google Play at every start: when a newer version exists, Google's own update window opens inside the app (the standard "in-app update" of Google Play; the App Store equivalent on iPhone later), and the person updates without leaving the app. The server, too, accepts only the newest version: an older app cannot save anything until it is updated. After the update the new terms of use are asked, if they changed.
7. The server asks for a new version only when Google has published it to everyone: every hour the server reads the version live on Google Play (a version still in review or rolled out to part of the people does not count). Nobody is stopped before Google can give them the update.

If something goes wrong

- Stop the rollout in the Play Console, or put the previous verified version back online (*Promote verified APK and server*).
- Block one or more faulty versions on every phone at once (Actions → *App versions*): the app offers only the update. Data stay.

14. Disaster recovery

What happens if data is lost or the infrastructure disappears: which backups exist, how far back we can go, how long it takes to be back, what we commit to and what we do not. The step-by-step runbook, with every manual action, is [docs/operations/disaster-recovery.md](#); the Infrastructure as Code is in the folder `DR/` of the repository.

What the service levels rest on: Cloudflare, Workers Free plan (today)

Cloudflare, Workers Free	What it means for HINT 365
No availability guarantee (no SLA): uptime commitments exist only on Cloudflare's paid business plans	we cannot promise a percentage of availability; HINT 365 is offered "as is", as the terms of use say
D1 Time Travel: the database can be put back to any minute of the last 7 days (30 days on Workers Paid, 5 US\$ a month)	minute-level recovery within 7 days, at no cost
Daily limits: 100,000 requests; D1 5 million rows read and 100,000 rows written; above them requests fail until midnight UTC (stored data is not affected)	far above today's use; if ever reached the service pauses until midnight UTC: Workers Paid removes the limit
Data bound to the EU (D1 jurisdiction eu)	databases and backups stay in the European Union
No daily copies kept by Cloudflare beyond Time Travel	we make our own: one encrypted backup every night, 30 kept

Backups: what exists and how often

Backup	How often	How far back	Where
Time Travel (Cloudflare, automatic, always on)	continuous, every change	any minute of the last 7 days	Cloudflare D1, EU
Nightly backup of both databases, encrypted (AES-256) with a phrase only the owner knows	once a night, 01:40 UTC	the last 30 nights	third database personalhealthy-vault , EU
Copy of every acceptance of the terms	at once, and checked every night	kept as proof	database personalhealthy-backup , EU

The backups are never in the GitHub repository (health data, public repository): only the code that makes and restores them is.

If data is lost now, what can be recovered?

Data as it was...	Possible?	With
a few minutes ago, a few hours ago, yesterday	yes, to the minute	Time Travel
last night · 3 days ago · a week ago	yes	Time Travel (up to 7 days) or the backup of that night
2 or 3 weeks ago	yes, as it was that night	the backup of that night
more than 30 days ago	no	nothing is kept longer, as the privacy policy says

Our commitments (internal targets)

What happened	RPO: data lost at most	RTO: back within	How
wrong data written, a mistaken deletion, noticed within 7 days	a few minutes	5 minutes	Time Travel
a database lost or broken	24 hours (last night)	30 minutes	rebuild + restore of last night's backup
the server broken	nothing	15 minutes	Build (or the previous verified version)
the whole infrastructure rebuilt from zero (same Cloudflare account)	24 hours	1 hour	Disaster recovery → rebuild, then the manual steps of the runbook
the Cloudflare account itself lost	all data (the backups are in the same account)	1 hour for an empty service; + Google's review if the address changes	until a copy outside the account exists (planned at the Google Play launch)

These are targets, not a contract: they depend on Cloudflare, GitHub and Google, which give no availability guarantee on the plans in use.

How a recovery runs

1. **By hand** (only if the Cloudflare account is new): sign in, keep the subdomain `comevava76`, create the API token, put it in GitHub; check the GitHub secrets (list in the runbook).
2. **Pipeline** Actions → *Disaster recovery* → `rebuild`, confirm `RESTORE`: creates the three databases in the EU, the tables, takes last night's backup, checks its fingerprint, decrypts it, puts the data back, shows the counts, and starts *Build*.
3. **Pipeline** *Build*: secrets into the server, server and Web Dashboard online, signed app, download check.
4. **By hand**, only what applies: Anthropic (a new AI key and its spend limit), Google (the sign-in client, only if the address changed), Google Play (the server's access), the owner's phone (owner code).
5. **By hand**, the check: health address, sign in on the app, readings and lab results there, Web Dashboard, Admin numbers, a new backup made.

Proof that it works

On the 1st of every month the pipeline runs a **drill** by itself: it builds a complete copy of the databases from zero, restores last night's backup into it, compares the counts with production, measures the time and deletes the copy. First drill on the real account, 02.10.2026: rebuilt from zero and restored in 24 seconds, same counts as production (1 account, 39 readings, 29 acceptances); the copy was then deleted.

15. Costs: what is free, and up to where

As published by the providers in October 2026; the limits can change: check their pricing pages before relying on them.

Service	Free up to	Above that	For HINT 365
Cloudflare Workers (the server)	100,000 requests a day; short computing time per request	Workers Paid, 5 US\$ a month (10 million requests a month included)	a person makes a few dozen requests a day: free well beyond 1,000 people
Cloudflare D1 (the database)	500 MB per database, 5 GB in total; 5 million rows read and 100,000 rows written a day	included in Workers Paid, then by use	today about 0.3 MB; a person adds a few kB a month
GitHub (code, tests, releases)	the repository is public: GitHub Actions minutes are free and unlimited on standard runners; releases are free	—	free
Google sign-in	free	—	free
Google Play	25 US\$ once for the developer account	15% of each subscription	see costs and earnings
Anthropic (Scan)	not free: about 0.2 US cents a photo	stops at the ceiling the owner sets	trial at most 0.09 US\$ a person; subscribers pay 4.99 US\$ a year

Everything except the Scan is free today. The Admin console shows the database size against the free limit, and the money spent on the Scan month by month.

16. Where to find the rest

Document	File
Technical architecture, drawings, flows and the database diagram (9 pages)	docs/architecture/HINT-Architecture.pdf
User guides, Italian and English	docs/HINT-Guida-IT.pdf , docs/HINT-Guide-EN.pdf
Terms of use and privacy policy	docs/legal/HINT365-Terms.pdf , docs/legal/HINT365-Privacy.pdf
Compliance dossier and GDPR record	docs/compliance/HINT365-Compliance.pdf , docs/compliance/gdpr.md
Vulnerability process, in detail	docs/security/vulnerability-management.md , docs/security/vulnerability-flow.pdf
Owner's costs and earnings	docs/business/HINT365-Owner-Economics.pdf
Errors and problems	docs/operations/observability.md

17. Picking the work up again, months later

Everything needed to continue is in the repository on GitHub: the code, the documents, and a diary of every working session. No memory of the chat is needed.

1. **Open an AI coding assistant** that can work on GitHub: Claude Code ([claude.ai/code](#), where this project was built) or another one such as Codex/ChatGPT. Old chats may still be there, but do not rely on them: the repository is the reference.
2. **Give it the repository:** [github.com/comevava76-web/personalhealthy](#).
3. **Paste this request:**

"Read Checkpoint/HANDOVER.md, AGENTS.md, CLAUDE.md and docs/HINT365-How-it-works.pdf. Tell me in a few lines what HINT 365 is, which version is online, what was done last and what was left open. Then check the error log and the security issues. Do not change anything until I say so."

4. **Read its summary** and decide the next feature or fix, in your own words.
5. The assistant follows the rules already written: change on a branch, documents updated, tests green (section 12), your "go", release (section 13): the download link, Google Play, and in future the Apple App Store (an iPhone version would be a new app, built separately, with the same server).
6. At the end of each session it adds a dated entry to [Checkpoint/HANDOVER.md](#), so the next one starts from there.

To remember	Where
What was done, session by session, and what is open	Checkpoint/HANDOVER.md
The rules of the project (in Italian)	CLAUDE.md , AGENTS.md
How to set up keys and accounts from zero	README.md
All documents, who updates them and when	docs/README.md
Accounts you own	GitHub (code), Cloudflare (server and database), Google Cloud and Google Play (sign-in, store), Anthropic (Scan), the password manager with <code>OWNER_CODE</code>